



PYTHON PROGRAMMING-PILOT (355)

NATIONAL 2022

PRODUCTION PORTION:

Program 1: Character Information _____ (400 points)

TOTAL POINTS _____ (400 points)

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for scoring.

1. Create a folder on the flash drive provided using your Member ID as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is scored as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be scored.

Assumptions to make when taking this assessment:

- The user will *not* enter invalid input.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All functions (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).
- If you create a class, then you must document the class and its methods.

Character Information

You are tasked with writing a program that processes video game character information using classes. The base class (*Character*) keeps data attributes for the following pieces of information:

- Character name
- Character id_number

The program must also have a class named *Hero* that is a subclass of the *Character* class. The *Hero* class should keep data attributes for the following information:

- level (1, 2, 3, 4, 5)
- loot (formatted with two decimal points)

Additionally, as with most games, there will be a boss at the end of each level. Program a *Boss* class that is a subclass of the *Character* class. The *Boss* class should keep data attributes for the following information:

- level (1, 2, 3, 4, 5)
- hp (integer)
- attack_damage (integer) (damage inflicted on hero)
- lifespan (NOTE: the number of attacks the boss can withstand by dividing the hp by 300)

Once you have written the classes, write a program that does the following:

1. Creates an object of the *Hero* class and prompts the user to enter data for Boss A (see Table 1).
2. Creates an object of the *Boss* class and prompts the user to enter data for Boss A (see Table 1).
3. Stores the data in the respective objects.
4. Retrieve and display the data on the screen from the respective objects.

Attribute	Hero A	Boss A
Name	Knight	Ogre
Character ID	111	333
Level	1	2
Loot	50.00	
Hit Points (hp)		5500
Attack Damage		99
Lifespan		

Requirements:

1. You must create a python program called `characterinformation.py`.
2. You must create the *Character*, *Hero*, and *Boss* classes within `characters.py`
3. Your program must create the Hero object.
4. Your program must prompt the user to enter in the correct Hero attributes for Hero A.
5. Your program must create the Boss object.
6. Your program must prompt the user to enter in the correct Boss attributes for Boss A.
7. Your program must retrieve and display the stored data from the respective objects to the user.
8. Your program must properly calculate the Boss's lifespan.
9. Your Member ID must appear as a comment at the top of the main source code file.
10. Your program must be commented appropriately.

Example Input/Output:

```
Hero Data Entry:
Enter the hero name: Knight
Enter the character ID number: 111
Enter the hero level: 1
Enter the hero loot value: 50

Boss Data Entry:
Enter the boss name: Ogre
Enter the character ID number: 333
Enter the boss level: 2
Enter the boss hp: 5500
Enter the boss attack damage: 99

Hero information:
Name: Knight
ID number: 111
Level: 1
Loot: $50.00

Boss information:
Name: Ogre
ID number: 333
Level: 2
HP: 5500.0
Attack Damage: 99.0
Lifespan: 18.33 attacks
```

Solution and Project

The project is present on the flash drive _____ 10 points

The project contains character.py and characterinformation.py _____ 10 points

Program Execution

The program runs/compiles from the USB flash drive _____ 10 points

If the program does not execute, then the remaining items in this section receive a score of zero.

The program prompts the user to enter in the correct Hero attributes for Hero A. _____ 10 points

The program prompts the user to enter in the correct Boss attributes for Boss A. _____ 10 points

The program properly retrieves and displays the Hero A data. _____ 30 points

The program properly retrieves and displays the Boss A data. _____ 30 points

The program properly calculates Boss A's lifespan. _____ 50 points

Data is properly displayed (ie. lifespan with two decimals points). _____ 20 points

Source Code Review

A comment containing the Member ID is present _____ 10 points

Functions and code sections are commented _____ 20 points

Character class is created with appropriate accessor and mutator methods _____ 50 points

Hero class is created with appropriate accessor and mutator methods _____ 50 points

Boss class is created with appropriate accessor and mutator methods _____ 50 points

Input and output code is properly formatted. _____ 30 points

Code uses a consistent variable naming convention _____ 10 points

Total Points = _____ / 400 points

NOTE TO GRADERS:

Use the following data to test the application.

Attribute	Hero B	Boss B
Name	Archer	Chimera
Character ID	222	444
Level	2	3
Loot	99.99	
Hit Points (hp)		12000
Attack Damage		250

Testing Output:

```
Hero Data Entry:
Enter the hero name: Archer
Enter the character ID number: 222
Enter the hero level: 2
Enter the hero loot value: 99.99

Boss Data Entry:
Enter the boss name: Chimera
Enter the character ID number: 444
Enter the boss level: 3
Enter the boss hp: 12000
Enter the boss attack damage: 250

Hero information:
Name: Archer
ID number: 222
Level: 2
Loot: $99.99

Boss information:
Name: Chimera
ID number: 444
Level: 3
HP: 12000.0
Attack Damage: 250.0
Lifespan: 40.00 attacks
```

character.py source example

Character Information Classes
MemberID

```
class Character:
    def __init__(self, name, id_number):
        self.__name = name
        self.__id_number = id_number

    def set_name(self, name):
        self.__name = name

    def set_id_number(self, id_number):
        self.__id_number = id_number

    def get_name(self):
        return self.__name

    def get_id_number(self):
        return self.__id_number

class Hero(Character):
    def __init__(self, name, id_number, level, loot):
        # Call superclass __init__ method.
        Character.__init__(self, name, id_number)

        # Initialize the level and loot attributes.
        self.__level = level
        self.__loot = loot

    # Mutator functions for level and loot.
    def set_level(self, level):
        self.__level = level

    def set_pay_rate(self, __loot):
        self.__loot = loot

    # Accessor functions for level and loot.
    def get_level(self):
        return self.__level

    def get_loot(self):
        return self.__loot

class Boss(Character):
    def __init__(self, name, id_number, level, hp, attack_damage):
        # Call superclass __init__ method.
        Character.__init__(self, name, id_number)

        # Initialize the level, hp, attack damage, lifespan attributes.
```

```
self.__level = level
self.__hp = hp
self.__attack_damage = attack_damage
self.__lifespan = 0

# Mutator functions for level, hp, attack damage, lifespan.
def set_level(self, level):
    self.__level = level

def set_hp(self, hp):
    self.__hp = hp

def set_attack_damage(self, attack_damage):
    self.__attack_damage = attack_damage

def set_lifespan(self, lifespan):
    self.__lifespan = lifespan

# Accessor functions for level, hp, attack damage, lifespan
def get_level(self):
    return self.__level

def get_hp(self):
    return self.__hp

def get_attack_damage(self):
    return self.__attack_damage

def get_lifespan(self):
    return (self.__hp / 300)
```

characterinformation.py source example

```
# Character Information Program
# MemberID

import character

def main():
    # Local variables
    hero_name= ""
    hero_id = ""
    hero_shift = 0
    hero_pay = 0.0

    boss_name= ""
    boss_id = ""
    boss_level = 0
    boss_hp = 0.0
    boss_attack_damage = 0.0
    boss_lifespan = 0.0

    # Get Hero data attributes
    print ('Hero Data Entry:')
    hero_name = input('Enter the hero name: ')
    hero_id = input('Enter the character ID number: ')
    hero_level = int(input('Enter the hero level: '))
    hero_loot = float(input('Enter the hero loot value: '))

    # Create an instance of Hero
    hero = character.Hero(hero_name, hero_id, hero_level, hero_loot)

    # Get Boss data attributes.
    print ("")
    print ('Boss Data Entry:')
    boss_name = input('Enter the boss name: ')
    boss_id = input('Enter the character ID number: ')
    boss_level = int(input('Enter the boss level: '))
    boss_hp = float(input('Enter the boss hp: '))
    boss_attack_damage = float(input('Enter the boss attack damage: '))

    # Create an instance of Boss.
    boss = character.Boss(boss_name, boss_id, boss_level, boss_hp, boss_attack_damage)

    # Display Hero information
    print ("")
    print ('Hero information:')
    print (f'Name: {hero.get_name()}')
    print (f'ID number: {hero.get_id_number()}')
    print (f'Level: {hero.get_level()}')
    print (f'Loot: ${hero.get_loot():.2f}')
```

```
# Display Boss information.
print("")
print('Boss information: ')
print(f'Name: {boss.get_name()}')
print(f'ID number: {boss.get_id_number()}')
print(f'Level: {boss.get_level()}')
print(f'HP: {boss.get_hp()}')
print(f'Attack Damage: {boss.get_attack_damage()}')
print(f'Lifespan: {boss.get_lifespan():.2f} attacks')

# Call the main function.
if __name__ == '__main__':
    main()
```